

Sistema de Gestão Hospitalar

Base de Dados

Henrique Moreira (119809)

Miguel Santos (119649)

(P6G2)

3/06/2026

INDICE

Contenido

Introdução 3
Requisitos Funcionais 4
Entidades 5
Diagramas 6
Queries SQL (DDL) 8
Queries SQL (DML) 10
Consultas e Vistas (SQL Views) 11
SQL UDF 12
SQL Stored Procedures 13
SQL Triggers 15
Interface 17
Conclusão 18

Introdução

Neste projeto, é apresentado o processo de modelação e desenvolvimento de um Sistema de Gestão Hospitalar. O principal propósito deste sistema é permitir o acompanhamento detalhado de todo o percurso do paciente. Este fluxo abrange desde a marcação inicial de avaliações clínicas até à realização de cirurgias, assegurando simultaneamente a correta alocação dos profissionais de saúde aos seus respetivos departamentos.

A arquitetura da base de dados foi estruturada de forma lógica e acessível, com o objetivo de facilitar o trabalho diário de médicos, enfermeiros e pessoal administrativo. Através desta solução, os utilizadores terão a capacidade de gerir as suas agendas, registar diagnósticos e consultar o histórico médico dos pacientes de forma eficiente e segura. Em suma, este relatório documenta a construção de uma solução centralizada e robusta para simplificar a gestão hospitalar.

Com esta abordagem, procura-se melhorar a experiência dos utentes, potenciar a eficiência dos profissionais de saúde e garantir uma organização estruturada da administração do hospital.

Requisitos Funcionais

Entidades	Funcionalidades
Paciente	- Pedir marcação/cancelamento de avaliações - Consultar histórico médico
Médico	- Consultar agenda - Consultar paciente - Realizar avaliações - Registar diagnósticos - Participar em cirurgias - Consultar histórico médico
Enfermeiro	- Consultar agenda - Consultar paciente - Assistir em cirurgias - Consultar histórico médico
Administrador	- Consultar agenda - Consultar paciente - Gerir médicos/enfermeiros - Gerir Inventário - Gerir Equipamento Médico - Gerir Salas - Gerir Avaliações - Gerir Cirurgias

Entidades

Para desenhar a base de dados do nosso Sistema de Gestão Hospitalar, o primeiro passo foi identificar os elementos do mundo real sobre os quais precisamos de guardar informação. A estas peças fundamentais chamamos Entidades.

Pessoa: Trata-se da entidade mais genérica do sistema.

Paciente: Representa o utente do hospital e é uma instância de Pessoa.

Médico: É o profissional de saúde responsável pelas avaliações, sendo uma instância de Pessoa que pertence a um departamento.

Enfermeiro: Representa o profissional de enfermagem, sendo também uma instância de Pessoa alocada a um departamento.

Administrador: É o responsável pela gestão hospitalar, figurando como uma instância de Pessoa que pertence a um departamento.

Hospital: Representa a própria instituição hospitalar à qual os vários departamentos pertencem.

Departamento: Representa as diferentes alas ou especialidades médicas do hospital, pertencendo a um hospital específico.

Tipo de Sala: Serve para classificar as salas do hospital de acordo com a sua finalidade, como, por exemplo, gabinete, bloco operatório ou enfermaria.

Sala: Representa os gabinetes ou salas físicas do hospital, que estão associadas a um departamento e a um tipo específico.

Equipamento Médico: Representa toda a maquinaria e os recursos físicos médicos, como aparelhos de Raio-X ou ecógrafos.

Sala_Equipamento: É uma tabela associativa utilizada para registar que equipamentos estão alocados a cada sala, incluindo a respetiva quantidade.

Avaliação: Representa um evento clínico agendado, que cruza a informação de um paciente e de um médico numa data e sala determinadas.

Diagnóstico: É o resultado clínico que deriva de uma avaliação, contendo a sua designação e a data em que foi realizado.

Cirurgia: Trata-se de um procedimento cirúrgico que tem origem num diagnóstico, sendo realizado numa sala de operações sobre um paciente.

Enfermeiro_Cirurgia: Tabela associativa que permite registar quais os enfermeiros que assistem a uma determinada cirurgia.

Medico_Cirurgia: Tabela associativa que guarda o registo dos médicos que realizam cada cirurgia.

Diagramas

Diagrama Entidade-Relacionamento: Ilustra o modelo conceptual do sistema.

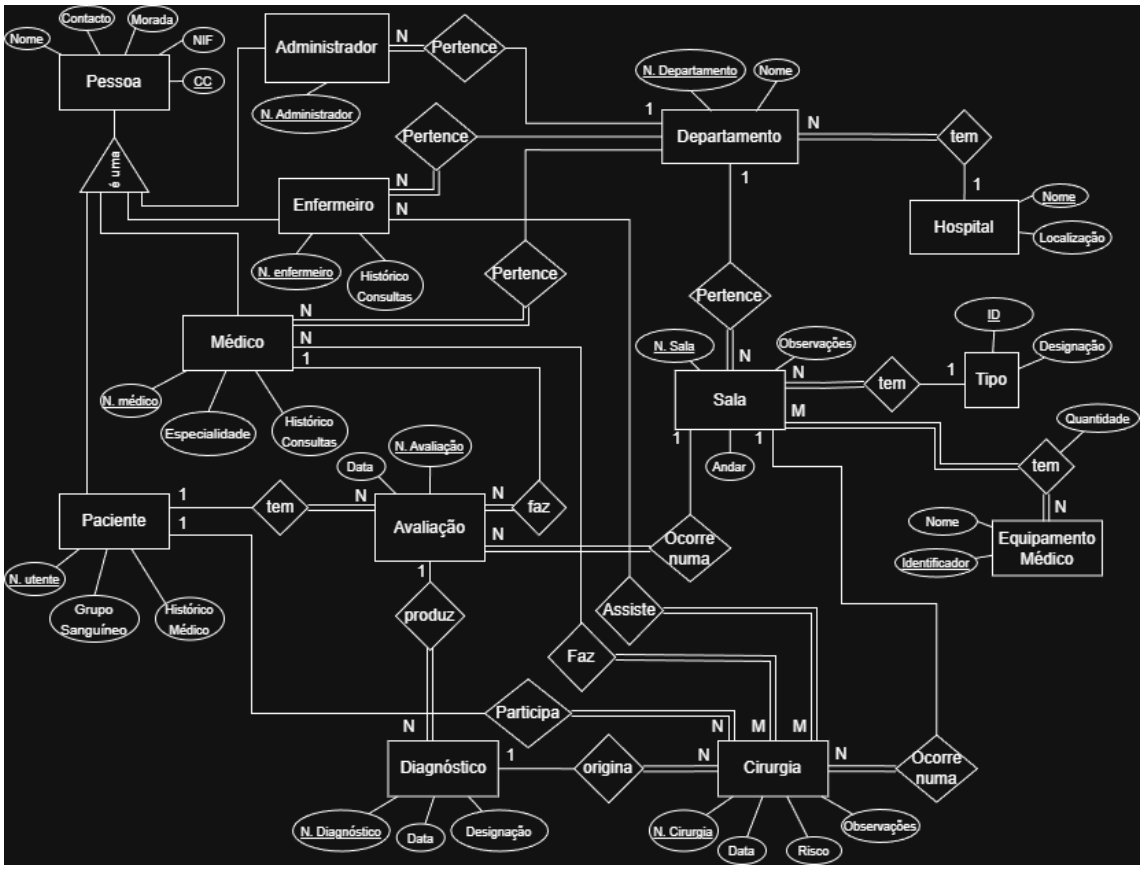


Diagrama Relacional : Traduz o modelo conceptual numa estrutura lógica de tabelas , democstra de forma direta como as ligações são feitas na prática através das chaves primárias e estrangeiras

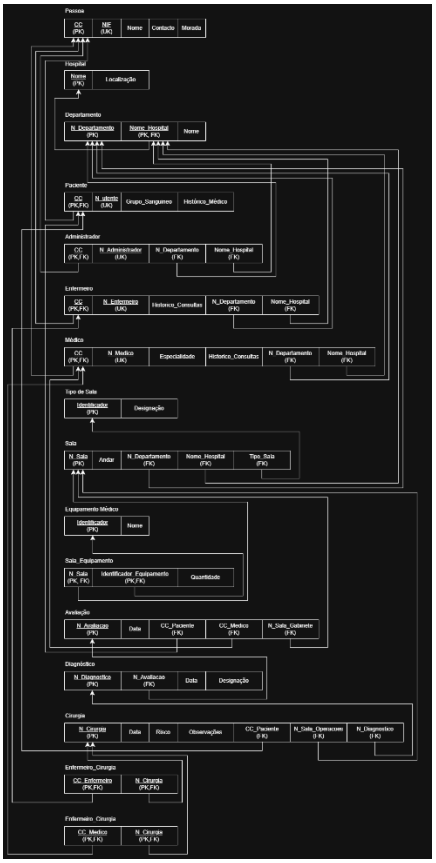
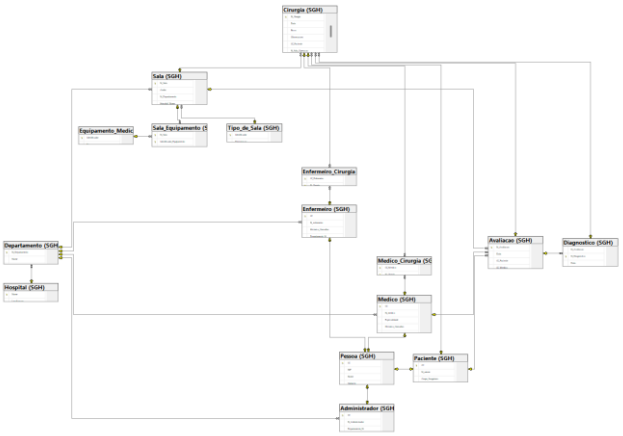


Diagrama SGBD : Abaixo está ilustrado o esquema relacional criado pelo SGBD utilizado.



Queries SQL (DDL)

Para construir a arquitetura física do nosso Sistema de Gestão Hospitalar, utilizámos comandos da linguagem SQL (DDL - Data Definition Language). O objetivo desta fase foi traduzir o nosso Diagrama Relacional para o servidor, criando as tabelas e estabelecendo as regras de integridade (como as Chaves Primárias e Estrangeiras) que garantem que os dados não contêm erros.

Todos os *scripts* DDL completos referentes à criação do esquema da base de dados e da totalidade das suas 16 tabelas encontram-se disponíveis no ficheiro anexado (SGH.sql)

abaixo os quatro exemplos mais representativos da lógica que aplicámos :

1) Entidade Base e Restrições Únicas :

A tabela Pessoa é a base estrutural de todos os perfis humanos do sistema. Aqui, definimos o CC como Chave Primária (PRIMARY KEY), garantindo que cada pessoa tem um identificador único. Adicionámos também uma restrição UNIQUE ao NIF, assegurando que não podem existir dois registos com o mesmo número fiscal.

```
-- ============================================================
-- 1. PESSOA
--    Entidade mais genérica do sistema.
-- ============================================================
CREATE TABLE SGH.Pessoa (
    CC          VARCHAR(20)     NOT NULL,
    NIF         VARCHAR(20)     NOT NULL,
    Nome        VARCHAR(100)    NOT NULL,
    Contacto    VARCHAR(50),
    Morada      VARCHAR(200),
    CONSTRAINT PK_Pessoa PRIMARY KEY (CC),
    CONSTRAINT UK_Pessoa_NIF UNIQUE (NIF)
);
```

2) Herança e Dependências :

A tabela Medico exemplifica perfeitamente o conceito de herança e dependência. O CC atua simultaneamente como Chave Primária e Chave Estrangeira (FOREIGN KEY), ligando o médico aos seus dados pessoais genéricos na tabela Pessoa. Adicionalmente, criámos uma ligação obrigatória ao Departamento, garantindo que um médico só pode ser inserido se estiver alocado a um departamento existente.

```sql
-- ============================================================
-- 7. MÉDICO
--    Profissional de Saúde responsável pelas consultas.
--    Instância de Pessoa.
-- ============================================================
CREATE TABLE SGH.Medico (
    CC                  VARCHAR(20)     NOT NULL,
    N_medico            VARCHAR(20)     NOT NULL,
    Especialidade       VARCHAR(100),
    Historico_Consultas TEXT,
    Departamento_N      INT             NOT NULL,
    Hospital_Nome       VARCHAR(100)    NOT NULL,
    CONSTRAINT PK_Medico PRIMARY KEY (CC),
    CONSTRAINT FK_Medico_Pessoa FOREIGN KEY (CC)
        REFERENCES SGH.Pessoa(CC),
    CONSTRAINT FK_Medico_Departamento FOREIGN KEY (Departamento_N, Hospital_Nome)
        REFERENCES SGH.Departamento(N_Departamento, Hospital_Nome),
    CONSTRAINT UK_Medico_N UNIQUE (N_medico)
);
```

3) Gestão de Eventos Complexos

A tabela Cirurgia é um dos pilares do fluxo hospitalar. Este bloco de código demonstra como cruzamos informação vital: para uma cirurgia acontecer, o sistema obriga a que exista um Paciente válido, uma Sala (física) onde o procedimento vai decorrer, e um Diagnóstico prévio que justifique a operação.

```sql
-- ============================================================
-- 14. CIRURGIA
--     Procedimento cirúrgico originado por um diagnóstico,
--     realizado numa sala de operações sobre um paciente.
-- ============================================================
CREATE TABLE SGH.Cirurgia (
    N_Cirurgia      INT             NOT NULL,
    Data            DATE            NOT NULL,
    Risco           VARCHAR(50),
    Observacoes     TEXT,
    CC_Paciente     VARCHAR(20)     NOT NULL,
    N_Sala_Operacao INT             NOT NULL,
    N_Diagnostico   INT             NOT NULL,
    CONSTRAINT PK_Cirurgia PRIMARY KEY (N_Cirurgia),
    CONSTRAINT FK_Cirurgia_Paciente FOREIGN KEY (CC_Paciente)
        REFERENCES SGH.Paciente(CC),
    CONSTRAINT FK_Cirurgia_Sala FOREIGN KEY (N_Sala_Operacao)
        REFERENCES SGH.Sala(N_Sala),
    CONSTRAINT FK_Cirurgia_Diagnostico FOREIGN KEY (N_Diagnostico)
        REFERENCES SGH.Diagnostico(N_Diagnostico)
);
```

4) Tabelas Associativas (Relações Muitos-para-Muitos)

Por fim, a tabela Sala_Equipamento é um exemplo de como resolvemos situações em que uma sala pode ter vários equipamentos, e o mesmo tipo de equipamento pode estar em várias salas. Utilizámos uma Chave Primária Composta (juntando o número da sala e o identificador do equipamento) para gerir o inventário e as respetivas quantidades de forma eficiente.

```sql
-- ============================================================
-- 11. SALA_EQUIPAMENTO
--     Tabela associativa que regista quais equipamentos
--     estão alocados a cada sala e em que quantidade.
-- ============================================================
CREATE TABLE SGH.Sala_Equipamento (
    N_Sala                  INT     NOT NULL,
    Identificador_Equipamento   INT     NOT NULL,
    Quantidade              INT     NOT NULL DEFAULT 1,
    CONSTRAINT PK_Sala_Equipamento PRIMARY KEY (N_Sala, Identificador_Equipamento),
    CONSTRAINT FK_SalaEquip_Sala FOREIGN KEY (N_Sala)
        REFERENCES SGH.Sala(N_Sala),
    CONSTRAINT FK_SalaEquip_Equipamento FOREIGN KEY (Identificador_Equipamento)
        REFERENCES SGH.Equipamento_Medico(Identificador)
);
```

Queries SQL (DML)

Após a definição da estrutura física da base de dados, o passo seguinte foi o insert das tabelas com dados fictícios para testar e validar o sistema. Para este efeito, utilizámos a linguagem de manipulação de dados SQL DML, especificamente a instrução INSERT INTO.

O aspeto mais importante nesta etapa foi respeitar rigorosamente a ordem de inserção. Devido às restrições das Chaves Estrangeiras (Foreign Keys), não podemos inserir um dado que dependa de outro que ainda não existe. Por isso, o povoamento foi estruturado em fases lógicas.

Todos os comandos de inserção completos, que garantem o insert de teste de todas as tabelas, encontram-se disponíveis no ficheiro anexado (inserts.sql).

Abaixo, apresentamos alguns exemplos representativos de cada fase:

Fase 1: Tabelas sem dependências

Começámos por registar as entidades que não dependem de nenhuma outra para existir. Neste caso, as infraestruturas principais .

```
-- 2. HOSPITAL (Nome: PK)
-- ============================================================
INSERT INTO SGH.Hospital (Nome, Localizacao) VALUES
('Hospital de Braga',      'Sete Fontes, São Victor, Braga'),
('Hospital de Guimarães', 'Rua dos Cutileiros, Creixomil, Guimarães');
```

Fase 2: Tabelas com dependência.

Com os hospitais criados, já pudemos criar as especialidades Departamentos .

```
-- 5. DEPARTAMENTO (PK composta: N_Departamento + Hospital_Nome)
--    FK → Hospital(Nome)
-- ============================================================
INSERT INTO SGH.Departamento (N_Departamento, Nome, Hospital_Nome) VALUES
(1, 'Cardiologia', 'Hospital de Braga'),
(2, 'Ortopedia',   'Hospital de Braga'),
(3, 'Pediatria',   'Hospital de Braga'),
(1, 'Urgência',    'Hospital de Guimarães'),
(2, 'Neurologia',  'Hospital de Guimarães');
```

Fase 3 : Fluxo clinico ,Por fim, com toda a estrutura e pessoal registados, testámos os eventos clínicos diários. Uma cirurgia é o exemplo perfeito de cruzamento de dados, pois obriga a que exista previamente um doente, uma sala de operações e um diagnóstico.

```
INSERT INTO SGH.Cirurgia (N_Cirurgia, Data, Risco, Observacoes, CC_Paciente, N_Sala_Operacao, N_Diagnostico) VALUES
(1, '2025-01-25', 'Alto',  'Ablação por cateter para correção de arritmia',       '1', 3, 1),
```

Consultas e Vistas (SQL Views)

Na nossa aplicação (Windows Forms), optámos por executar a maioria das consultas SQL diretamente no código (ex: SELECT Nome, Localizacao FROM SGH.Hospital).

Ainda assim, para simplificar a leitura de dados frequentes ou que exigem o cruzamento de várias tabelas, criámos as seguintes Vistas (Views):

1. Vista de Pessoas (vw_Pessoas)

Simplifica o acesso aos dados genéricos essenciais de todos os utilizadores.

```
CREATE VIEW SGH.vw_Pessoas
AS
SELECT
    CC,
    NIF,
    Nome,
    Contacto,
    Morada
FROM SGH.Pessoa;
```

2. Vista de Pacientes (vw_Pacientes)

Utiliza um INNER JOIN para unir os dados clínicos do Paciente com o seu Nome (guardado na tabela Pessoa). Isto permite à aplicação ler toda a informação do utente num único passo.

```
CREATE VIEW SGH.vw_Pacientes
AS
SELECT
    p.CC,
    ps.Nome,
    p.N_utente,
    p.Grupo_Sanguineo,
    p.Historico_Medico
FROM SGH.Paciente p
INNER JOIN SGH.Pessoa ps ON ps.CC = p.CC;
```

SQL UDF

Para evitar a repetição de código e otimizar o desempenho do sistema, criámos UDFs (User Defined Functions) que encapsulam lógicas de negócio e cálculos recorrentes. Destacamos as duas funções principais:

1. Verificação de Salas (fn_SalaDisponivel)

Garante que não existem sobreposições de agendamentos. Cruza as tabelas de Avaliações e Cirurgias (usando UNION ALL) e devolve 1 se a sala estiver livre, ou 0 se estiver ocupada naquela data.

```
CREATE OR ALTER FUNCTION SGH.fn_SalaDisponivel
    (@nSala INT, @data DATE)
RETURNS BIT
AS
BEGIN
    DECLARE @ocupada INT;
    SELECT @ocupada = COUNT(*) FROM (
        SELECT N_Sala_Gabinete AS Sala, Data FROM SGH.Avaliacao
        UNION ALL
        SELECT N_Sala_Operacao, Data FROM SGH.Cirurgia
    ) AS eventos
    WHERE Sala = @nSala AND Data = @data;

    RETURN CASE WHEN @ocupada = 0 THEN 1 ELSE 0 END;
END;
GO
```

2. Estatísticas de Médicos (fn_TotalAvaliacoesMedico)

Uma função de agregação rápida criada para apoiar a gestão hospitalar. Recebe o Cartão de Cidadão de um médico e devolve instantaneamente o número total de avaliações que este já realizou.

```
CREATE OR ALTER FUNCTION SGH.fn_TotalAvaliacoesMedico
    (@cc VARCHAR(20))
RETURNS INT
AS
BEGIN
    RETURN (
        SELECT COUNT(*) FROM SGH.Avaliacao WHERE CC_Medico = @cc
    );
END;
GO
```

SQL Stored Procedures

Para automatizar ações repetitivas, garantir a integridade dos dados e centralizar a lógica de negócio na própria base de dados, desenvolvemos diversos Stored Procedures. A utilização destes procedimentos permite que a aplicação de Windows Forms apenas necessite de chamar uma rotina pelo nome, enviando os parâmetros necessários, sem ter de conhecer a complexidade interna das tabelas.

Destacamos abaixo dois exemplos estruturais que demonstram técnicas avançadas de segurança e gestão de dependências:

1. Transações e Controlo de Erros (sp_TransferirMedico)

Ao transferir um médico de departamento ou de hospital, é fundamental garantir que a operação não falha a meio. Para isso, implementámos um bloco estruturado de TRANSACTION com tratamento de erros. Desta forma, garantimos a propriedade de Atomicidade: ou o médico é transferido com sucesso e as alterações são gravadas , ou, caso ocorra uma falha de sistema, a base de dados reverte para o estado original , evitando dados inconsistentes.

```
CREATE OR ALTER PROCEDURE SGH.sp_TransferirMedico
    @cc            VARCHAR(20),
    @novoDep       INT,
    @novoHospital  VARCHAR(100)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;
    BEGIN TRY
        UPDATE SGH.Medico
        SET Departamento_N = @novoDep,
            Hospital_Nome  = @novoHospital
        WHERE CC = @cc;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        THROW;
    END CATCH;
END;
GO
```

2. Eliminação Segura em Cascata (sp_RemoverMedico)

Devido às rigorosas restrições de Chaves Estrangeiras do nosso sistema (SQL DDL), não é possível apagar um médico com um simples comando DELETE. Se o médico tiver consultas associadas, o SQL bloqueia a ação.

Para resolver isto de forma limpa na aplicação, este Stored Procedure encarrega-se de limpar metodicamente o histórico do médico, "de baixo para cima". Primeiro elimina os Diagnósticos associados às suas Avaliações, depois as próprias Avaliações, retira o médico do registo de Cirurgias e, só no final, remove o registo principal na tabela Medico.

```sql
CREATE OR ALTER PROCEDURE SGH.sp_RemoverMedico
    @cc VARCHAR(20)
AS
BEGIN
    SET NOCOUNT ON;
    -- Diagnósticos dependem de Avaliações que dependem do médico
    DELETE FROM SGH.Diagnostico
    WHERE N_Avaliacao IN (
        SELECT N_Avaliacao FROM SGH.Avaliacao WHERE CC_Medico = @cc
    );
    DELETE FROM SGH.Avaliacao        WHERE CC_Medico = @cc;
    DELETE FROM SGH.Medico_Cirurgia  WHERE CC_Medico = @cc;
    DELETE FROM SGH.Medico           WHERE CC = @cc;
END;
GO
```

SQL Triggers

Os Triggers são blocos de código que são executados de forma automática em resposta a eventos específicos na base de dados, como um INSERT, UPDATE ou DELETE. No nosso sistema, utilizámo-los principalmente para garantir a integridade das regras de negócio e automatizar a atualização de dados.

Abaixo destacamos 2 exemplos que refletem diferentes tipos de validação e automatização:

1. Validação Lógica de Infraestrutura (trg_Avaliacao_ValidarTipoSala)

Para prevenir erros humanos no agendamento, este trigger atua sempre que é feita uma nova marcação ou alteração de uma Avaliacao. Ele verifica automaticamente o tipo de sala associada. Se o sistema detetar que tentaram marcar uma simples consulta médica (Avaliação) num Bloco Operatório, a operação é imediatamente bloqueada e é devolvido um erro.

```
CREATE OR ALTER TRIGGER SGH.trg_Avaliacao_ValidarTipoSala
ON SGH.Avaliacao
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (
        SELECT 1
        FROM inserted i
        JOIN SGH.Sala s ON s.N_Sala = i.N_Sala_Gabinete
        JOIN SGH.Tipo_de_Sala t ON t.Identificador = s.Tipo_Sala
        WHERE t.Designacao LIKE '%Bloco%' OR t.Designacao LIKE '%Operat%'
    )
    BEGIN
        RAISERROR('Uma avaliação não pode ser agendada numa sala de bloco operatório.', 16, 1);
        ROLLBACK TRANSACTION;
    END;
END;
GO
```

2) Prevenção de Eliminação Perigosa :

Este é um exemplo de um trigger INSTEAD OF DELETE. Antes de permitir a eliminação de um paciente, o sistema verifica se ele tem algum histórico de cirurgias. Caso tenha, a eliminação é bloqueada para proteger o histórico médico do hospital. Se não tiver cirurgias, o trigger assume o controlo e apaga o paciente em segurança.

```
CREATE OR ALTER TRIGGER SGH.trg_Paciente_ImpedirRemocaoComCirurgias
ON SGH.Paciente
INSTEAD OF DELETE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (
        SELECT 1 FROM SGH.Cirurgia c
        JOIN deleted d ON d.CC = c.CC_Paciente
    )
    BEGIN
        RAISERROR('Não é possível remover um paciente com cirurgias registadas. Remova as cirurgias primeiro.', 16, 1);
        RETURN;
    END;
    -- Se não tiver cirurgias, permite a remoção
    DELETE FROM SGH.Avaliacao  WHERE CC_Paciente IN (SELECT CC FROM deleted);
    DELETE FROM SGH.Paciente   WHERE CC           IN (SELECT CC FROM deleted);
END;
GO
```

Para facilitar a avaliação e a replicação do projeto no servidor do professor, todo o código auxiliar aqui detalhado englobando Stored Procedures, Triggers e UDFs foi compilado e centralizado num único ficheiro script denominado

(UDF_SP_TRIGGERS_Everything_in_one.sql.)

Interface

Para a interação do utilizador final com a base de dados, foi desenvolvida uma aplicação Desktop em C# utilizando a tecnologia Windows Forms. Embora o foco central do projeto seja a modelação de dados, a camada de interface foi construída para garantir a segurança e a integridade do sistema:

Prevenção de SQL Injection: Em todas as operações de manipulação de dados a comunicação com a base de dados é feita exclusivamente através de SqlParameter. Isto previne vulnerabilidades de injeção de código malicioso.

Gestão de Exceções e Restrições Físicas: A aplicação utiliza blocos try-catch para intercetar erros gerados pelo SQL Server. Por exemplo, se um utilizador tentar apagar um Administrador ou um Departamento que possua registos dependentes (violando as restrições de Foreign Key), a interface impede o bloqueio do programa (crash) e apresenta uma mensagem de aviso ao utilizador.

Visualização Complexa (Master-Detail): Para processos complexos como o agendamento de Cirurgias, a interface gere múltiplas tabelas associativas em simultâneo. O utilizador consegue, num único ecrã, definir os detalhes da operação e gerir dinamicamente os médicos e enfermeiros alocados a esse procedimento.

Conclusão

O desenvolvimento deste Sistema de Gestão Hospitalar revelou-se uma experiência enriquecedora que permitiu aplicar na prática os conceitos de modelação e administração de bases de dados, desde a análise inicial de requisitos e diagramação (DER e ER) até à implementação física estruturada (SQL DDL e DML).

A integração de mecanismos avançados como Views, Stored Procedures, Triggers e UDFs foi crucial para automatizar regras de negócio vitais e garantir a integridade da informação, evitando falhas como a sobreposição de salas ou a perda indevida de histórico clínico.

Por fim, a ligação bem-sucedida a uma interface gráfica em Windows Forms comprovou que uma base de dados implementada é o pilar central de qualquer aplicação segura, culminando num sistema capaz de dar resposta às exigências diárias de uma infraestrutura hospitalar real.

A distribuição das tarefas e a respetiva contribuição :

Henrique Moreira (119809) : 40%

Miguel Santos (119649) : 60%